tmdoc::tmdoc 0.20.1 – Tcl Markdown processor tutorial

Detlef Groth, University of Potsdam, Germany

2026-07-08

---

filter-julia - filter-octave - filter-python - filter-r - filter-tcl
filter-abbrev - filter-mtex - filter-tcrd
tmdoc-tutorial

Abstract

In the last years reproducible research has gained focus from the research community. It is a type of literate programming, introduced by Donald Knuth in 1984, where code of a programming language is embedded within standard text documents, like LaTeX, Markdown, AsciiDoc or Typst documents. The code is evaluated and adds or replaces it's own code with the code output. Alternatively the code chunks as well as the output result can be also hidden if needed.

There exist several tools which allow this type of programming/documenting, for instance for the R programming language Knitr or Sweave. Although Knitr is an engine for dynamic report generation especially for the statistical language R it supports as well other programming languages such as for instance Python 3, Perl, Haskell, Ruby and more than 20 others with a few limitations.

Embedding of Tcl in the Knitr framework is somehow difficult as it has no command line option to evaluate Tcl code on the fly. There might be workarounds by creating a Tcl command line application which allows this. Form my perspective however, the Tcl programming as a mature highly dynamic scripting language invented by John Ousterhout in 1988, should not need a huge framework to support literate programming with Tcl itself. Therefore the decision to write a Tcl program to implement a literate programming package for Tcl just based on standard Tcl which is called *tmdoc::tmdoc*, or short just *tmdoc*. In its current state *tmdoc* supports standard code chunks with options to display and hide code and code output, to embed inside text code evaluations and image creation. This basic subset of a literate programming environment should be sufficient to create nice technical and statistical reports, tutorials about Tcl packages and other types of documentation.

Table of Contents

- Introduction
- Color Codes
- Standard Code Blocks
- The puts Statement
- Inline Codes
- Error Handling
- Creating Tables
- Including Files
- Writing Images
- Pandoc compatible code chunk syntax
- Knitr, R Markdown compatible syntax
- Other Command Line Tools
- Kroki Webservice Diagrams

- LaTeX Equations
- Chemical Equations
- Programming
- References
- Abbreviations
- Alerts
- CSV Display
- HTML Tags
- Comments
- Figure Numbering
- Statistical Reports
    - R Code Evaluation
    - Python Code Evaluation
    - Octave Code Evaluation
- Music Chord Sheets
- tblocks - diagrams from Markdown code
- Code Chunk Options
- Summary
- Installation
- See Also
- Processing Information

Introduction

The Tcl package `tmdoc::tmdoc` which is both a standalone command line application and as well a Tcl package is a report generator and a literate programming tool for Tcl. It can capture the results and plots or images from data analysis and works well with other packages like Tk, or gdtcl. The current text is a short tutorial on how to do literate programming with Tcl using the package. Literate programming is a style of progamming where in a text document, shorter or longer code fragments of a programming language are embedded. The document is then "weaved", the programming code is replaced with it's output. The produced document can thereafter be further processed to other formats.The `tmdoc::tmdoc` package supports Markdown, Typst, AsciiDoc, Quarkdown and LaTeX as documentation languages. Using markup processors and document converters like `pandoc`, `typst`, `asciidoctor` or the Tcl application mndoc the output of the `tmdoc` processor can be converted finally to other document formats like pdf or html.

To give an example, below is the pipleine which was used to create the HTML document you are currently viewing using https://github.com/mittelmark/tmdoc and https://github.com/mittelmark/mndoc :

```
$ tmdoc tmdoc-tutorial.tmd - | \
    mndoc - tmdoc-tutorial.html --css tmdoc.css
```

With the pandoc command line application yo can as well convert the Markdown output of *tmdoc* to many other formats. The pipeline is then the following:

```
      tmdoc         pandoc
TMD --------> MD ----------> HTML/PDF/DOCX
```

Now the mandatory "Hello World!" example. If you embed the following text into your Markdown document with the extension, `.tmd`, *tmdoc* will produce, after converting the document from Markdown to HTML via *mndoc* or *pandoc*, the following input (replace the leading single quote with a backtick, necessary here to avoid interpretation by *tmdoc*)

```
'``{.tcl}
puts "Hello World!"
'``
```

will produce this output:

```
puts "Hello World!"
```

```
Hello World!
```

Code block and/or the output can be as well hidden using chunk arguments, that will be described later in more detail. Short code fragments can be directly embedded into the text. Here an example for this approach:

```
The curent date and time is: `tcl clock format [clock seconds] -format "%Y-%m-%d %H:%M"`
```

This is the output:

The curent date and time is: 2026-10-29 22:14

Color Codes

Now follows a short tutorial for the features of the *tmdoc* Markdown processor. For the code blocks the following color codes are used, grey is example Tcl code which is not evaluated by Tcl, it shows mainly how the code chunks looks in the Markdown document, in creme is Tcl code which is evaluated, in blue is the output of the creme tcl code before. Red code lines indicate code errors.

So here again the color codes:

```
gray: code examples which are not evaluated within this document
```

```
creme: code examples which are evaluated by Tcl
```

```
blue: result output of evaluated Tcl code
```

```
red: errors resulting from Tcl code evaluations
```

Standard Code Blocks

Standard code block are started with three tickmarts at the beginning of a line and in curly brace the string tcl. Separated by a space might follow several evaluation options which will be described later in more detail.

Here the Tcl-Markdown input (replace the single quote with a backtick):

```
'``{tcl echo=true}
set x 1
set a 1
set a
set x
'``
```

Please note that the indentation is neccessary only for the example here, as tmdoc is processing tmdoc inside tmdoc. So in fact there should be no four spaces at the beginning in your code.

And here the resulting output:

```
set x 1
set a 1
set a
set x
```

```
==> 1
```

Only the last statement is returned. The arrow indicates the return value of the current code block. You should use *puts* if you need to output more than the last statement.

The puts Statement

You can have several puts statements in your code which are printed before the final return value. The channel stdout is displayed as well in the results part. Also the *-nonewline* option can be used:

```
puts "Hello World 1"
puts "Hello World 2"
set x 3
puts "Hello World $x"
puts stdout "Hello World [incr x]"
puts -nonewline "Hello"
puts " World [incr x]"
puts -nonewline stdout "Hello"
puts stdout " World [incr x]"
# let's reset x to one
set x 1
```

```
Hello World 1
Hello World 2
Hello World 3
Hello World 4
Hello World 5
Hello World 6
==> 1
```

You can as well write into file channels as usually:

```
set out [open file.ext w 0600]
puts $out "Hello File World!"
close $out
# now display the content
set filename file.ext
if [catch {open $filename r} infh] {
    puts stderr "Cannot open $filename: $infh"
    exit
} else {
    while {[gets $infh line] >= 0} {
        puts stdout "file: $line"
    }
}
```

```
file: Hello File World!
```

To supress any output, your last statment should return an empty string:

```
set a 5
set a ""
```

Alternatively you can supress the output at all by giving an option `results=hide` to the code chunk

````
'``{.tcl results=hide}
set a 1
'``
````

```
set a 1
```

There is now no output with 1 here to see as we used chunk option `results=hide`.

You can as well supress both showing the Tcl code and the output of the Tcl code by specifying `results=hide` and `echo=false` as shown in the following example:

````
'``{tcl results=hide,echo=false}
set a 2
'``

Our variable a has now a value of `tcl set a` although we don't know why
as the programming code is hidden.
````

Our variable a has now a value of 2 although we don't know why as ...

This feature can be useful if you don't want to show in a final version of a text document how the computation was done, but only the results of this computation within the text, for instance to display date and used Tcl packages as can be seen at the very end of this document.

Inline Codes

````
Let's look what is the value of x. The value of x is `tcl set x`. Is it one??
````

Let's look what is the value of x. The value of x is 1. Is it one??

Yes it is!

Now let's try two times inline code on one line:

````
The value of x is: `tcl incr x` and now `tcl incr x`!.
````

The value of x is: 2 and now 3!. So also this works. Don't write too long Tcl statements directly in the text. You should perform your major computations in the code blocks. Please note as well, that the inline code chunks can't wrap over multiple lines. So opening and closing backtick must be on the same line.

Error Handling

Let's display errors of code blocks:

```
set y 2
set z
```

```
can't read "z": no such variable
    while executing
"set z"
```

Should given an error ... It does!

Now check for inline code errors in text:

```
This line contains an inline code error `tcl set z`!!
```

This line contains an inline code error *??can't read "z": no such variable??*!!

Creating Tables

Since *tmdoc* version 0.7.0 there is as well support for tables using the `results=asis` option. Here an example:

```
'``{.tcl results=asis,echo=false}
set h [list A B C]
set d [list 1 2 3 4 5 6 7 8 9]
puts [list2mdtab $h $d]
'``
```

A	B	C
1	2	3
4	5	6
7	8	9

Including Files

Sometimes we store information which should be available for a set of files in separate file to reuse this information in other files. Examples for this are headers and footers or a table of contents which is generated by tmdoc. Let's create such a file here and then include that file. Let's assume we have a file lipsum.txt which contains Lorem lipsum text. We can then easily include that file like this:

```
Some text - indented and emphasized.

> _`tcl include lipsum.txt`_

Some more text.
```

Here the output:

Some text - indented and emphasized.

> *Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.*

Some more text.

Since version 0.17 the user of tmdoc can as well omit the tcl prefix of the inline chunk like this:

```
Some text - indented and bold.

> __`include lipsum.txt`__

Some more text.
```

Which will produce the following output:

Some text - indented and bold.

> **Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.**

Some more text.

Since version 0.18.1 you can as well add specific code, for instance HTML tags like pre or div which should be displayed before and after the included file. Here an example which would display an C source code file within a pre HTML environment.

```
`tcl include sample.c {<pre> </pre>}`
```

That syntax is useful if you like to provide specific arguments for the environment like adding a class name:

Error: file 'sample.c' does not exists!

Since version 0.18.2 you can as well shorten *pre* blocks without classnames like this:

```
`tcl include assets/hello2.c pre`
```

Here the output:

```
#include <stdio.h>
int main() {
    int x = 1;
    int y = 1;
    if (x == 1 && y == 1) {
        printf("All looks good!");
    }
    printf("Hello 2nd C World!\n");
    return(0);
}
```

Sometimes you would like to include as well files within code chunks and then process these files within the code chunk environment for this you can use the `#INCLUDE "filename"` directive as it is know for the C programming language. We use an uppercase `#INCLUDE` to not confuse C programming. Here an example (remove the leading space before the # char):

```
'``{.tcrd echo=false}
#INCLUDE "assets/music.txt"
'``
```

And here the output:

```
      Am      C
Hello Include World!
        G
This is Music.
```

Here an example for an included C file which will be then compiled:

```
'``{.shell cmd="gcc %i -o hello2&&./hello2 > %o" chunk.ext=c ext=txt}
#INCLUDE "assets/hello2.c"
'``
```

And here the output:

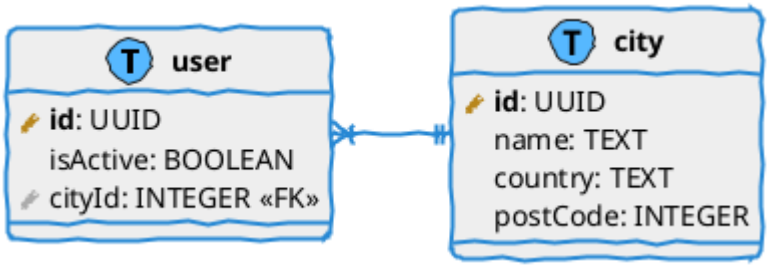
```
#include <stdio.h>
int main() {
    int x = 1;
    int y = 1;
    if (x == 1 && y == 1) {
        printf("All looks good!");
    }
    printf("Hello 2nd C World!\n");
    return(0);
}
```

```
All looks good!Hello 2nd C World!
```

Now as an example let's include a PlantUML file which is then processed with the plantuml command line application.

```
'``{.shell cmd="plantuml -tpng %i" echo=false}
#INCLUDE "assets/table.pml"
'``
```

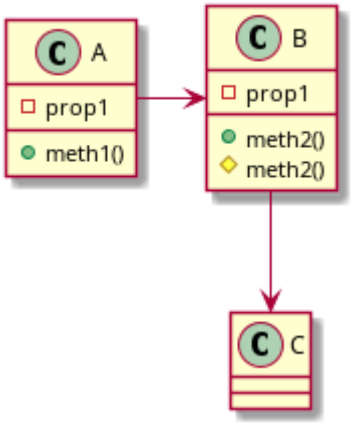
And here the output:



Now let's use the kroki web service instead:

```
'``{.kroki dia="plantuml",echo=FALSE}
#INCLUDE "assets/class.pml"
'``
```

And here the output:



Writing Images

Displaying images generated by Tcl code is as well possible. Here an example using the *tsvg* package which is embedded within the tmdoc application:

```
lappend auto_path ..
package require tsvg
tsvg set width 100
tsvg set height 100
# Tcl like syntax without hyphens
tsvg circle cx 50 cy 50 r 45 stroke black stroke-width 2 fill salmon
tsvg text x 29 y 45 Hello
tsvg text x 27 y 65 World!
tsvg write hello-world.svg
### cleanup
tsvg set code ""
```



To allow automatic embedding of the current image you have to create a figure command like this:

```
proc figure {filename} {
    tsvg write $filename
    tsvg set code ""
    return ""
}
```

Here an example:

````
```{.tcl fig=true echo=false results=hide}
tsvg circle -cx 50 -cy 50 -r 45 -stroke black -stroke-width 2 -fill skyblue
tsvg text -x 29 -y 45 Hello
tsvg text -x 23 -y 65 World2!
```
````

And here the output, only the image should be displayed:



Pandoc compatible code chunk syntax

Since tmdoc version 0.5 as well Pandoc compatible code chunk options are supported. For instance spaces after the triple codes are ignored, and chunk options can be given space separated in addition to the comma separated approach.

Here an example:

```
'`` {.tcl}
set x 3
puts $x
'``
```

And here the output:

```
set x 3
puts $x
```

```
3
```

Now the options: `{.tcl echo=false results="hide"}`

```
set x 4
puts $x
```

Knitr, R Markdown compatible syntax

Knitr and R-Markdown are using R in the background, so in R strings must be quoted, code chunks arguments are separated by comma. The terms for booleans are TRUE and FALSE. So **tmdoc** as well supports this syntax. Here an example:

Here an example:

```
'``{r echo=TRUE,results="show",eval=TRUE}
print(R.version.string)
'``
```

And here the output:

```
print(R.version.string)
```

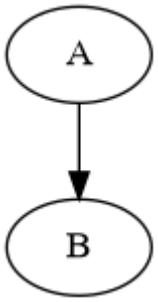
```
> print(R.version.string)
[1] "R version 4.5.0 (2025-04-11)"
```

Other Command Line Tools

Since version 0.8.0 there is as well support for embedding input for command line applications which produce graphics inside code chunks, for instance using tools like GraphViz dot, Pikchr or PlantUML or even R. To do this you add code chunks with the `.shell` tag and provide the command to process in the code chunk options the command to process the content with the `cmd` attribute. There are as well placeholders, two for the input filename `%i` or '%f', one for the basename of the input file, so the filename without the extension '%b' and one for the output filename: `%o`. Here an example for using the dot executable:

````
```{.shell cmd="dot -Tpng %i -o%o" echo=false}
digraph g {
   A -> B
}
```
````

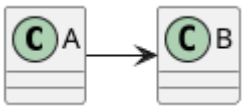
And here the output:



Let's continue with an example using the `plantuml` command line application. Here the output filename is guessed from the input filename, so there is no need to give the `%o` argument.

````
```{.shell cmd="plantuml -tpng %i" echo=false}
@startuml
class A { }
class B { }
A -> B
@enduml
```
````

And here the output:



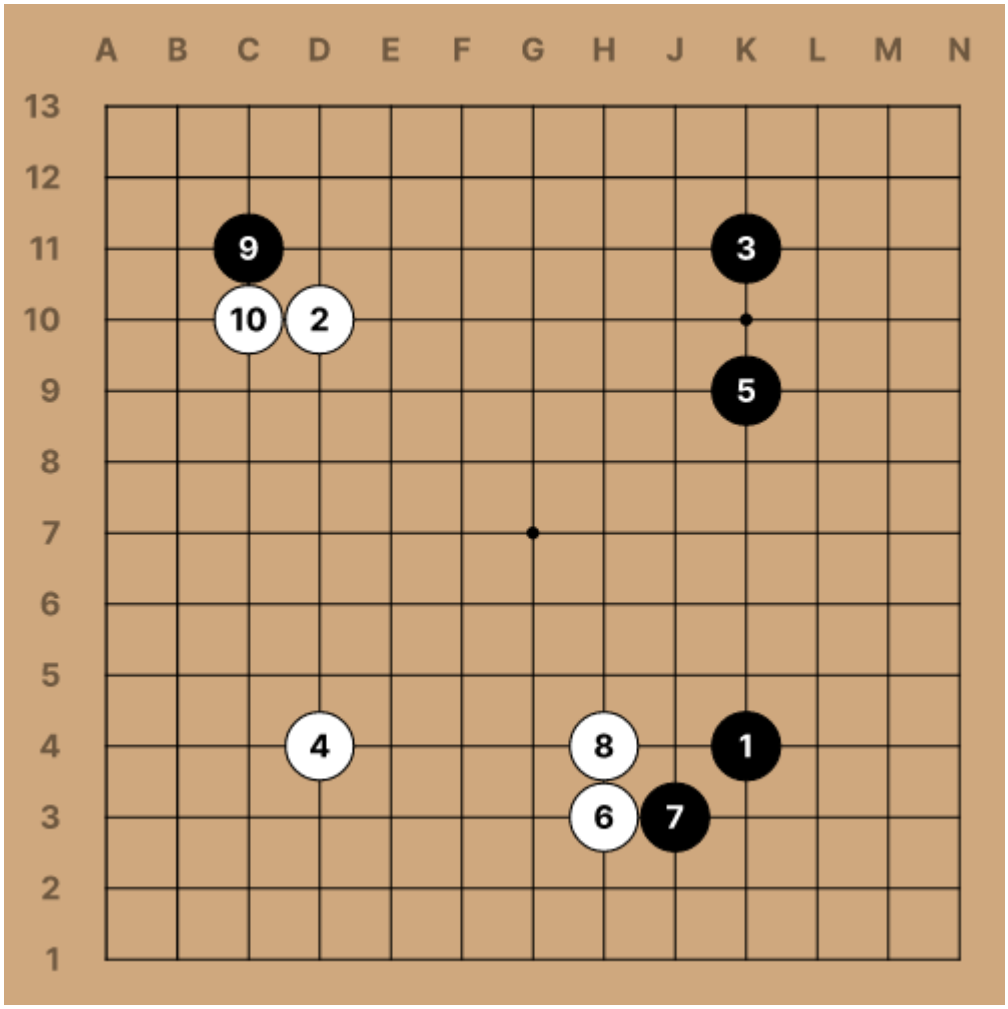
Some times you would like to use the same input presented in the code block before but with different command line arguments. In this case you can add as well the term LASTFILE on the first line of the code chunk to reuse this. Here an example to display different moments of a Go board game. So instead of writing again and again the same long game record in the code chunk you just write the short text LASTFILE into it. Here an example for this approach displaying the initial phase of a 13x13 board game but with different command line options for the display using the command line tool sgf-render:

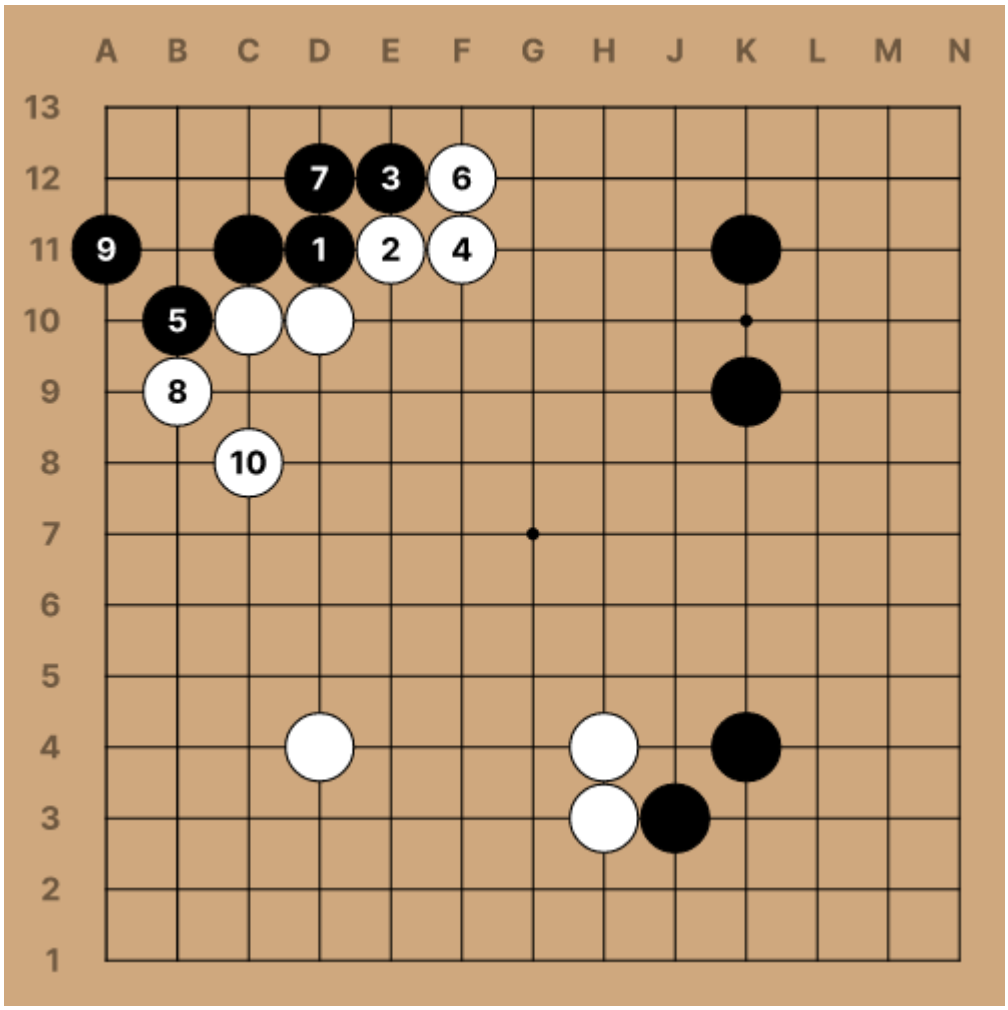
````
```{.shell cmd="sgf-render %i --format png --outfile %o --width 500
                    --move-numbers=1-10 -n 10" echo=false}
(;FF[4]
CA[UTF-8]
GM[1]
DT[2025-02-28]
GN[Freundschaftsspiel]
```
````

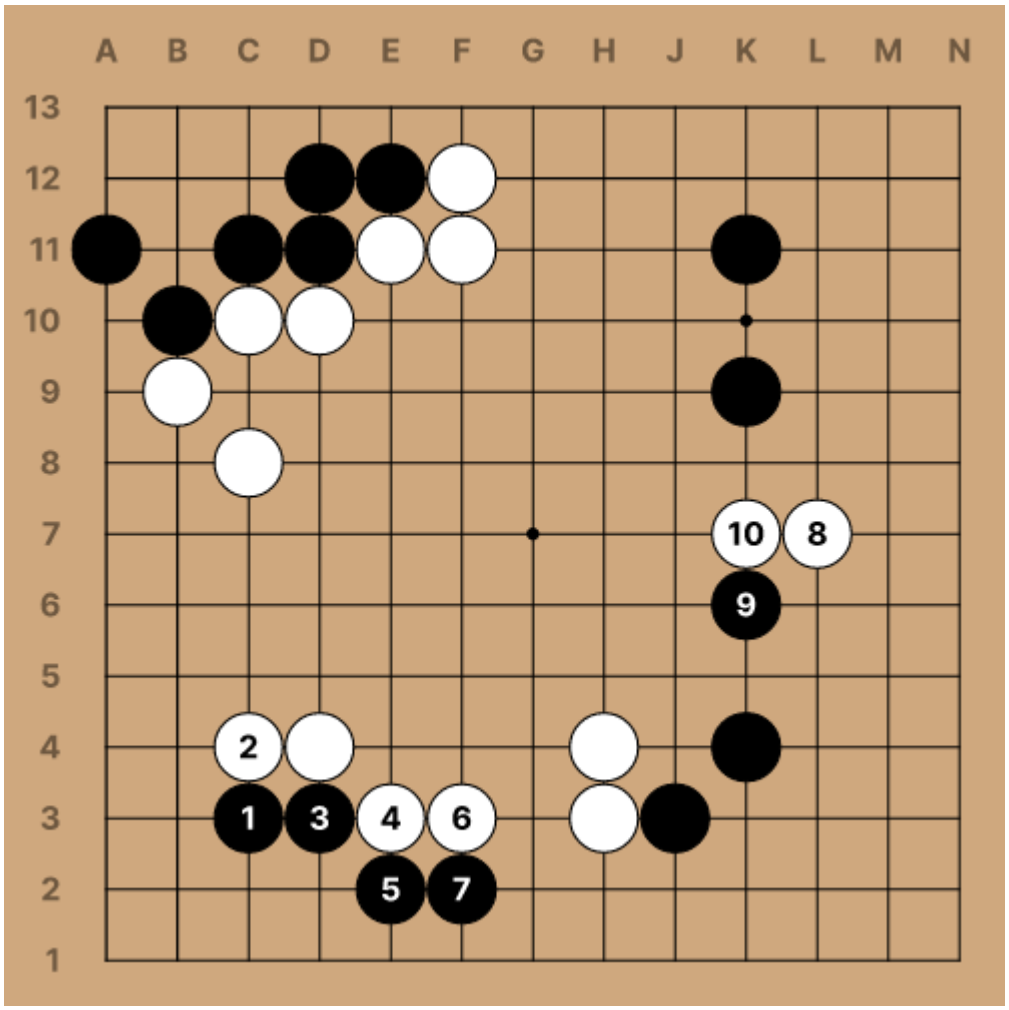
```
PB[Black Player]
PW[White Player]
BR[15k]
WR[14k]
:::
'``
'``{.shell cmd="sgf-render %i --format png --outfile %o --width 500
    --move-numbers=11-20 -n 20" echo=false}
LASTFILE
'``

'``{.shell cmd="sgf-render %i --format png --outfile %o --width 500
    --move-numbers=21-30 -n 30" echo=false}
LASTFILE
'``
```

And here the ouput (the sgf codes on top where shortened):




A B C D E F G H J K L M N
13 12 11 10 9 8 7 6 5 4 3 2 1
7 3 6
9 1 2 4
5
8
10

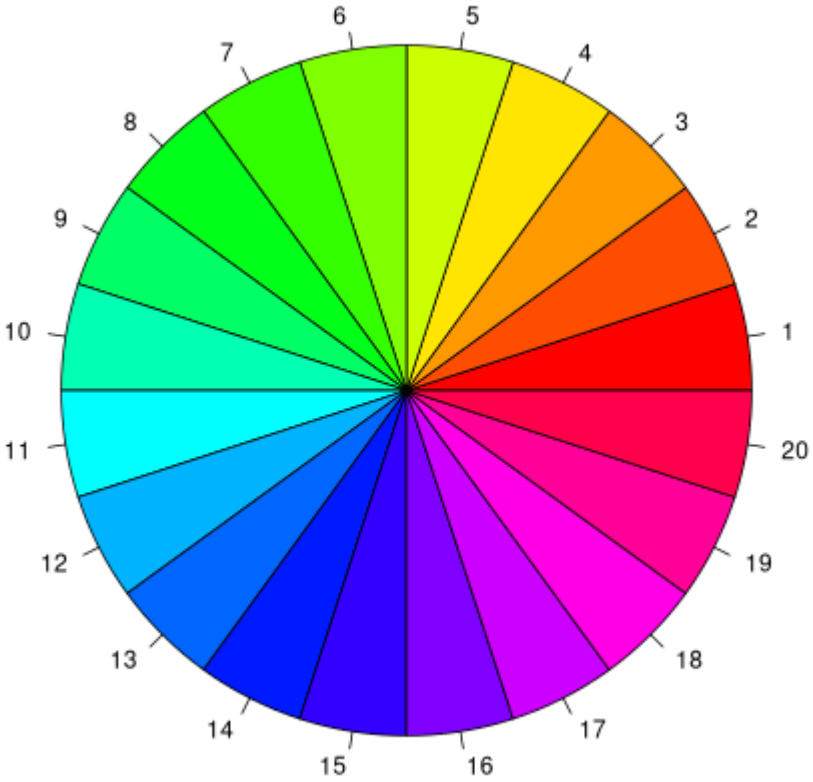


Let's continue with an example using the statistical programming language R:

```
'``{.shell cmd="Rscript %i %o"}
### outfile is the argument %o
outfile=commandArgs(trailingOnly=TRUE)[1]
png(outfile)
par(mai=rep(0.1,4))
pie(rep(1,20),col=rainbow(20))
dev.off()
'``
```

And here the output:

```
### outfile is the argument %o
outfile=commandArgs(trailingOnly=TRUE)[1]
png(outfile)
par(mai=rep(0.1,4))
pie(rep(1,20),col=rainbow(20))
dev.off()
```



To have compatibility with pantcl you can use as well cmd as the chunk indicator.

```
'``{.cmd cmd="dot -Tpng -o%o %i" echo=false}
digraph g {
  rankdir="LR"
  node[shape=box,style=filled,fillcolor=orange1]
  A -> B -> C
}
'``
```

Output:



Error handling is as well implented let's say you have an error in your GraphViz code, like a missing curly brace at the end:

```
digraph g {
    rankdir="LR"
    node[shape=box,style=filled,fillcolor=orange1]
    A -> B -> C
```

Error: Cannot open '/home/dgroth/workspace/tmdoc/modules/tmdoc/chunk-shell-29.png'! ```error Error: dot: can't open chunk-shell-29.txt while executing "exec {*}$cmd"

Error: dot: can't open chunk-shell-29.txt ```

Kroki Webservice Diagrams

The kroki.io webservice allows you to embed diagram code for many diagram tools into your websites in form of base64 encoding. Since version 0.9.0 *tmdoc* supports embedding of this using kroki code chunks. If the tool wget is installed on the users machine files will be downloaded and stored locally, these files are then used instead of the the web link. The download is only repeated if the code chunk content does change. Since version 0.14.2 firs tlocal installations of GraphViz (dot), PlantUML (plantuml) and Ditaa (ditaa) will be checked and if these tools are available locally as applications, they will be used instead of the webservice. So the Kroki webservice will be only used if the tools are not installed on the users machine.

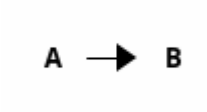
See here for some examples.

A simple Ditaa diagram can be created like this:

````
```{.kroki dia=ditaa fig.path=kroki ext=png}
A --> B
```
````

Here the output:

```
A --> B
```

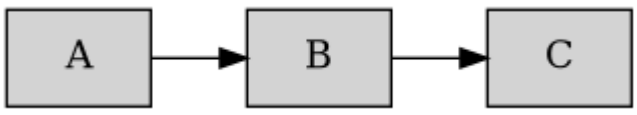


Here an example for a GraphViz Kroki diagram:

````
```{.kroki dia=graphviz fig.path=kroki ext=png}
digraph a {
   rankdir="LR"
   node[shape=box,style=filled]
   A -> B -> C
}
```
````

Here the output:

```
digraph a {
    rankdir="LR"
    node[shape=box,style=filled]
    A -> B -> C
}
```

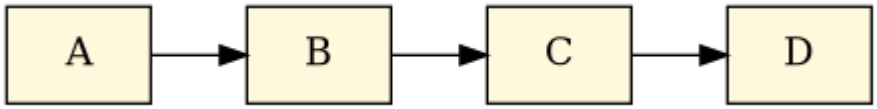


Using the option `echo=false` we can as well hide the code display:

````
```{.kroki dia=graphviz ext=png fig.path=kroki echo=false}
digraph a {
   rankdir="LR"
   node[shape=box,style=filled,fillcolor=cornsilk]
   A -> B -> C -> D
```
````

```
}
```

Here the result:



LaTeX Equations

Powered by CodeCogs

The https://editor.codecogs.com webservice allows you to embed LaTeX equations into your documents. The equation png file is downloaded and cached in the local file system, only if the equation is changed the file is downloaded again. Here an example:

```
'``{.mtex}
E = mc^2  \tag{1}
'``
```

Here the output:

```
E = mc^2  \tag{1}
```

$$E = mc^2 \tag{1}$$

Here an example for a matrix:

```
'``{.mtex}
\begin{bmatrix}
1 & 2 & 3 \\      \tag{2}
4 & 5 & 6
\end{bmatrix}
'``
```

and here the output again:

```
\begin{bmatrix}
1 & 2 & 3 \\      \tag{2}
4 & 5 & 6
\end{bmatrix}
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \tag{2}$$

Using the option `echo=false` you can hide the LaTeX code:

```
'``{.mtex echo=false}
\begin{bmatrix}
1 & 2 & 3 \\
4 & 5 & 6 \\  \tag{3}
5 & 6 & 7
\end{bmatrix}
'``
```

And here the output where you only should see the equation but not the LaTeX code as echo was set to false:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 5 & 6 & 7 \end{bmatrix} \tag{3}$$

Chemical Equations

Since version 0.19.0 tmdoc allows you as well to embed simple chemical equations with ce code chunks. Equations look like this:

```
Glucose creation:
`ce 6H2O + 6CO2 -> C6H12O6 + 6O2`

Respiration with backward arrow:
`ce 6CO2 + 6H2O <- C6H12O6 + 6O2`

The dissocation of water showing back and forward arrows is show here:
`ce 2H2O <-> H3O+ + OH-`

Dissociaton of Natrium Phosphate:
`ce Na3PO4 <-> 3Na+ + PO_4^3-`

`ce 2MnO4^-  3Mn^2+ + 4OH- <-> 5MnO2 + 2H2O`

Isotop: `ce ^14C`
```

Here the output:

Glucose creation: $6H_2O + 6CO_2 \rightarrow C_6H_{12}O_6 + 6O_2$

Respiration with backward arrow: $6CO_2 + 6H_2O \leftarrow C_6H_{12}O_6 + 6O_2$

The dissocation of water showing back and forward arrows is show here: $2H_2O \leftrightharpoons H_3O^+ + OH^-$

Dissociaton of Natrium Phosphate: $Na_3PO_4 \leftrightharpoons 3Na^+ + PO_4^{3-}$

$2MnO_4^- 3Mn^{2+} + 4OH^- \leftrightharpoons 5MnO_2 + 2H_2O$

Isotop: ^{14}C

So there are just three type of arrows as shown above. Furthermore, any number is lowered, any charge us uppered, text following a hat symbol is as well uppered. So that more than one positive or negative charged can be displayed.

> **Hint:** To show these numbers above each other you need to add a specific style section to your HTML document which manages this. This style is part of the default tmdoc.css style file. Here an example for the code you need to add to your own file:

```
<style>
/* chem span tag */
.chem {
  font-family: system-ui, sans-serif;
}

/* Normalize size and vertical positions */
```

```
.chem sub,
.chem sup {
  font-size: 0.75em;        /* smaller than base text */
  line-height: 0;           /* avoid extra line height */
}

span.pos {
  display: inline-table; /*contain floats*/
  vertical-align: middle;
  margin-left: -0px; /*no whitespace on i element*/
}
/* placement of  _N^N(-|+)  */
.pos sup, .pos sub {
  float: left;
  clear: left;
  font-size: 0.75em;
  line-height: 1;
}
.pos sup { margin-left: -15px; color: red;}
</style>
```

Further we can do a little bit of math like this about the ranking of the Pythagoreans means:

```
`ce m_a &ge; m_g &ge; m_h`
```

Here the output:

$m_a \geq m_g \geq m_h$

To simplify the use of LaTeX formulas for LaTeX command which have HTML entities with the same name, like for instance greek letters or these greater or equal symbols shown above we can even use the backslash version like so:

```
`ce m_a \ge m_g \ge m_h and \alpha and \beta and \gamma`
```

Here the output:

$m_a \geq m_g \geq m_h$ and α and β and γ

And here an example for the times and the entities:

```
`ce CV = 100 \times (SD \div Mean)`
```

And here the output:

CV = 100 × (SD ÷ Mean)

More of these commands which can be mapped from LaTeX to HTML can be found in the w3school unicode math reference. Just look there into the Entity column. If you need full LaTeX math support within your Markdown files, go to the LaTeX section of this tutorial.

Programming

Sometimes you would like to prepare programming tutorials where you embed your code with explanations in a Markdown document and you then like to test the code and show the output of the program. For this purpose as well `.shell` code chunks can be used. The command line should then contain the command line for compilation and separated with two && symbols the command to execute the program. Here a simple "Hello World!" script written in the Python programming language.

````
'``{.shell cmd="python3 %i > %o" chunk.ext=py ext=txt}
print("Hello Python World!")
'``
````

Here the output:

```
print("Hello Python World!")
```

```
Hello Python World!
```

Here an example for a C program:

````
'``{.shell cmd="gcc %i -o hello&&./hello > %o" chunk.ext=c ext=txt}
#include <stdio.h>
int main() {
    printf("Hello C World!\n");
    return(0);
}
'``
````

And here the output:

```
#include <stdio.h>
int main() {
    printf("Hello C World!\n");
    return(0);
}
```

```
Hello C World!
```

References

Since version 0.12.0 tmdoc as well supports the inclusion of references. References can be included like this:

```
This is a references of a paper by Author  [@Author2021].
```

The author key must be available in a BibTeX file which can be either provided in the document header in the Yaml section (since tmdoc 0.13.0) like this:

```
---
...
bibliography: filename.bib
...
---
```

Alternatively it can be provided within the text like this:

```
package require citer
citer::bibliography filename.bib
```

Here an example for a citation. Groth et. al. have written a nice PCA review [Groth2013]. In contrast Sievers et. al. described the Clustal Omega command line application to align multiple sequences [Sievers2011].

The list of citations can be then displayed using the tcl code 'tcl citer::bibliography'. Here the output:

References:

- **[Groth2013]** - Groth, D., Hartmann, S., Klie, S., Selbig, J. (2013). Principal Components Analysis. In: Computational Toxicology, Humana Press. 930: 527-547.
- **[Sievers2011]** - Sievers, F., Wilm, A., Dineen, D., Gibson, T. J., Karplus, K., Li, W., Lopez, R., McWilliam, H., Remmert, M., Soding, J., Thompson, J. D., Higgins, D. G. (2011). Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. Mol. Syst. Biol. 7: 539.

Abbreviations

Abbreviations can be defined in a separate Yaml file which should be mentioned in the document preamble like so:

```
---
...
abbreviations: abbrev.yml
...
```

Here the text:

```
This is an abbrev {DG} and this is another for the
Author: {author}. The document was created at {date}.
This an empty abbreviation {empty}.

Now an abbreviation which is in the `abbrev.yml` file
- tmdoc is: '{tmdoc}'!
```

And here the output:

This is an abbrev Detlef Groth, University of Potsdam, Germany and this is another for the Author: Detlef Groth, University of Potsdam, Germany. The document was created at 2026-07-08 16:53. This an empty abbreviation {empty}.

Now an abbreviation which is in the `abbrev.yml` file - tmdoc is: 'Tcl Markdown document processor'!

Even longer abbreviations can be displayed like:

```
> _{lipsum}_
```

Here the output:

> *Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.*

For Markup languages which do not support Yaml headers like Typst or AsciiDoc the Yaml file which should be used for abbreviations since version 0.15.0 can be as well given on the command line of tmdoc like this:

```
tmdoc file.ttyp file.typ --abbrev abbrev.yml
tpyst compile file.typ
```

The Tcl procedure `tmdoc::tmdoc` for this purpose has the `-abbrev filename.yml` argument.

Alerts

Since version 0.13.0 tmdoc as well supports Alerts for the Markdown output. Alerts are following the same syntax as the Github alerts:

Here an example:

```
> [!NOTE]
> This is the note text. Note note!!

> [!TIP]
> Do not overload the text with alerts like this!
```

Highlighting is completely managed by the stylesheet. The alerts are placed in a div tag with the name side-alertname where alertname is the lowercase version of the alert name. See the file tmdoc.css for a stylesheet based implementation. The advantage of this approach is that for instance *weasyprint* can properly convert these to the PDF format.

Here some examples. The alert type is always seen in bold, so that is what you need to write in the first alert line in brackets preceded with an exclamation mark:

> **Note:** This is the note text. Note note!!

> **Tip:** Do not overload the text with alerts like this!

> **Important:** That is very important! Or not??

> **Info:** This is an info, info, info! This is an info, info, info! This is an info, info, info! This is an info, info, info!!!

> **Hint:** This is a hint which you can ingnore. But you might need that hint to answer an exam questions.

Question: Be or not to be! That is the question!!

Warning: This is a warning, warn, warn! This is a warning, warn, warn! This is a warning, warn, warn! This is a warning, warn, warn!

Error: An error happend!

Caution: This might have consequences! An error can happen! Can we have a third line?

CSV Display

Since version 0.13.0 it is as well to embed CSV (Comma Separated Values) data into the documents and return the data as text. Here an example:

````
'```{.csv results=asis}
col1,col2,col3,col4
1,2,3,4
5,6,7,8
9,10,11,12
'```
````

And here the output:

```
col1,col2,col3,col4
1,2,3,4
5,6,7,8
9,10,11,12
```

col1	col2	col3	col4
1	2	3	4
5	6	7	8
9	10	11	12

As with other code chunks you can as well hide the input by using `echo=false`. Further column with no header could be introduced by using an empty space. Also Markdown styling is supported. Here data from an other table:

```
'```{.csv results=asis echo=false}
' ,col1,col2,col3,col4
'__R1__,11,12,13,14
'__R2__,15,16,17,18
'__R3__,19,20,21,22
'__R4__,23,24,25,26
'```
```

And here the output:

	col1	col2	col3	col4
R1	11	12	13	14
R2	15	16	17	18
R3	19	20	21	22
R4	23	24	25	26

If you values contain itself a separator like the comma you have to enclose the values within quotes like in the following examples:

```
,col1,col2,col3,col4
__R1__,"11, 12 or 13",12,13,14
__R2__,15,"16, 17 or 18",17,18
__R3__,19,20,"21, 22 or 23",22
__R4__,23,24,25,26
```

	col1	col2	col3	col4
R1	11, 12 or 13	12	13	14
R2	15	16, 17 or 18	17	18
R3	19	20	21, 22 or 23	22
R4	23	24	25	26

HTML Tags

Markdown only supports a subset of HTML tags. Sicen version 0.17.0 using the tmdoc package or the application can be used to insert Markdown un-supported HTML tags using the 'tcl tag TAGNAME TEXT CLASSNAME' syntax. For instance you can insert the 'kbd' tag like this:

Example:

```
In MicroEmacs as in Bash the key combination
`tcl tag kbd Ctrl` + `tcl tag kbd t` can be used to transpose two characters.
```

And here the output:

In MicroEmacs as in Bash the key combination <kbd>Ctrl</kbd> + <kbd>t</kbd> can be used to transpose two characters.

The styling is here provided by the *tmdoc.css* style sheet file. That style sheet support as well a menu class for the 'kbd' tag to show menu entries. Here an example.

Here an example:

```
Files in MicroEmacs can be either saved via the
`tcl tag kbd Ctrl-x`  `tcl tag kbd Ctrl-s` keys or via the menu entry
`tcl tag kbd File menu` &#8594; `tcl tag kbd Save menu`.
```

And here the output:

Files in MicroEmacs can be either saved via the `Ctrl-x` `Ctrl-s` keys or via the menu entry `File` → `Save`.

As this is a little bit tedious to write but I need that code often for my lectures **tmdoc** provides as well two short methods using 'kbd' and 'menu' for inline code injections. Here an example:

```
`kbd Ctrl-x` `kbd Ctrl-s` or  `menu File` &#8594; `menu Save` are saving a file
with MicroEmacs.
```

which translates to:

`Ctrl-x` `Ctrl-s` or `File` → `Save` are saving a file with MicroEmacs.

Here an other example using 'menu':

```
Short  code  snippets  are  configured  in Geany vs the  `menu  Tools` ->
`menu "Configuration&nsbp;Files"` -> `menu snippets.conf` menu entry.
entries.
```

And here the output:

Short code snippets are configured in Geany vs the `Tools` -> `Configuration Files` -> `snippets.conf` menu entry. entries.

Please note that, if you give a single string containing a space, you have to write the HTML code for a string there so ` `.

To make this even easier the user can as well give a list argument in curly braces like this:

```
That a file which does not has the _.md_ file extension is a Markdown file
can be configured in Geany using the:
`menu {Document "Set Filetype" "Markup Languages" "Markdown source file"}`
menu checkbox.
```

Here the output:

That a file which does not has the *.md* file extension is a Markdown file can be configured in Geany using the: `Document` → `Set Filetype` → `Markup Languages` → `Markdown source file` menu checkbox.

In this case you even do not need to know the HTML code for the arrow.

You can as well use the named HTML codes more easily like this:

Here an example (please note that you have to protect the semicolon with a backslash):

```
`kbd &crarr\;` might stand for an `kbd ENTER` key.
```

And here the output:

`↵` might stand for an `ENTER` key.

To look for more symbols you can use in this context have a look at the list of UTF-8 arrows and their HTML symbols.

Using the underline HTML tag 'u' you can as well highlight menu shortcuts like so:

```
That a file which does not has the _.md_ file extension is a Markdown file
can be configured in Geany using the (remove the newline next line):
`menu {<u>D</u>ocument "Set File<u>t</u>ype" "<u>M</u>arkup Languages"
   "<u>M</u>arkdown source file"}`
menu checkbox.
```

Here the output:

That a file which does not has the *.md* file extension is a Markdown file can be configured in Geany using the: `Document` → `Set Filetype` → `Markup Languages` → `Markdown source file` menu checkbox.

Comments

Since version 0.17.0 there is as well for true comments where the text is not only hidden but as well excluded from the output document.

Here is an example:

```
This is some text.

<!--
Some comment which should be exluded.
Some more text
-->

That should be visible again.
```

And here the output:

This is some text.

That should be visible again.

Figure Numbering

Since version 0.17.1 **tmdoc** as well supports automatic figure and table numbering. To number your figure or table you can use the syntax 'nfig label' or 'ntab label' where the label should only consist of letters or numbers. If you repeat a label then the initial figure numbering can be used. Here an example.

```
The first  figure is `nfig  first`.  The first  table is `ntab  first`. If you
repeat a label  somewhere  later in the text like for figure `nfig first`, then
simply the numbering is repeated. The same is true for `ntab first`. Whereas
the call for a with a not used label like in figure  `nfig  second`  the label
gets a new number. So you can use a figure or table label  before and after the
actual figure for referencing and displaying the correct number.
```

Here the output:

The first figure is 1. The first table is 1. If you repeat a label somewhere later in the text like for figure 1, then simply the numbering is repeated. The same is true for 1. Whereas the call for a with a not used label like in figure 2 the label gets a new number. So you can use a figure or table label before and after the actual figure for referencing and displaying the correct number.

Here an example to label a Ditaa image:

```
The figure `nfig rect` below shows some simple rectangles.

'``{.kroki echo=FALSE}
+----------------+       /----------------\    /\
|                |  -->  |  cEFF          |    \/
+----------------+       \----------------/
'``
**Figure `nfig rect`:** Sample rectangles in Ditaa syntax.
```

And here the output:

The figure 3 below shows some simple rectangles.



Figure 3: Sample rectangles in Ditaa syntax.

As you can see lines starting with Figure are rendered especially the same is true for tables. Let's show a table caption with automatic labeling.

Here an example:

```
**Table `ntab first`:** An example for a CSV table.

'```{.csv echo=FALSE}
,col1,col2,col3
__A__,1,2,3
__B__,4,5,6
__B__,7,8,9
'```
```

And here the output:

Table 1: An example for a CSV table.

	col1	col2	col3
A	1	2	3
B	4	5	6
B	7	8	9

Statistical Reports

R Code Evaluation

Beside Tcl, since version 0.14.0 we can as well embed R, Python and Octave and since version 0.16.0 as well Julia code chunks within our documents.
The code will be then evaluated step by step and the output of the commands in text form or as images can be displayed in the output document. That way it is possible to write statistical reports and scientific papers purely

in Markdown documents. Code chunks can be either embedded in code blocks within triple back tick code fences or inline in the text using single back-ticks. The difference of this approach presented below using pipes is that the status of the preceding code chunks can be used in the following ones. So the status of the application is cached allowing in the following code chunks reuse code and variables from the chunks before. In contrast the shell based approach presented before just isolates every code chunk as an independent piece of code.

Let' start with a R code chunk. Please note that in the code example with prefix the triple back-ticks with single quotes to avoid interpretation.

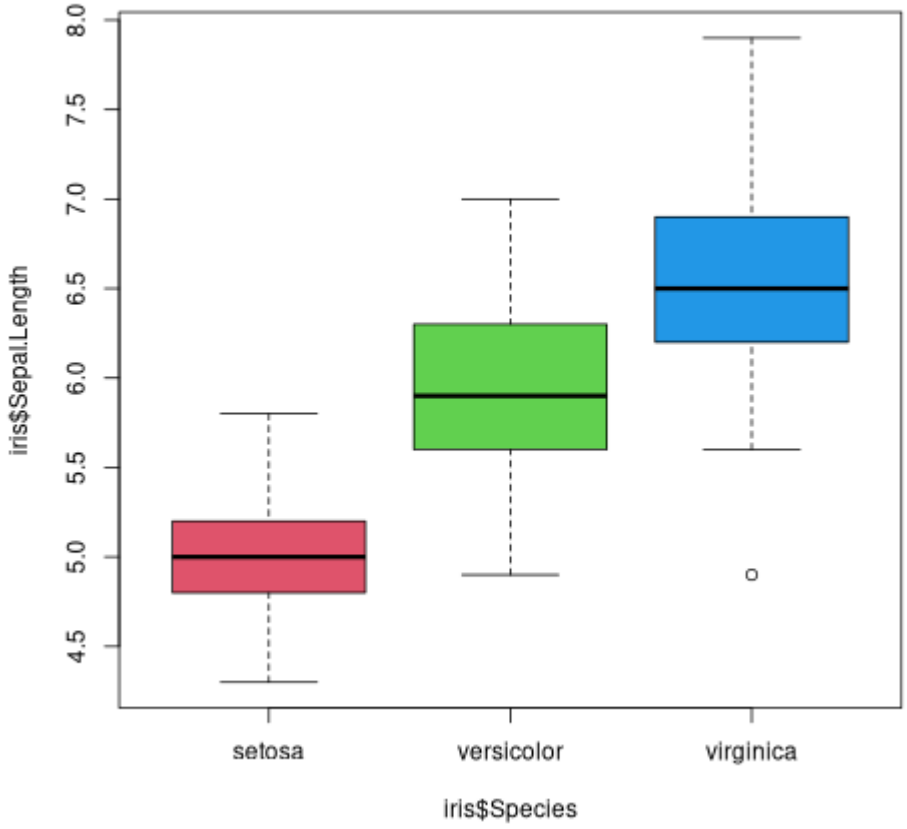
```
'```{.pipe pipe="R" results="show"}
data(iris)
png("testr.png")
boxplot(iris$Sepal.Length ~ iris$Species,col=2:4)
dev.off()
print(2)
x=3
'```

![testr.png](testr.png)
```

Here the output of this chunk:

```
data(iris)
png("testr.png")
boxplot(iris$Sepal.Length ~ iris$Species,col=2:4)
dev.off()
print(2)
x=3
```

```
> data(iris)
> png("testr.png")
> boxplot(iris$Sepal.Length ~ iris$Species,col=2:4)
> dev.off()
null device
          1
> print(2)
[1] 2
> x=3
```



As you can see, to create a figure we need to add the png and the `dev.off` functions and then add the image inclusion as Markdown command. This can e be little bit tedious. So instead using the option `fig=true` we can avoid adding the to code lines which open the figure and the print command to save the image and use the current chunk name.

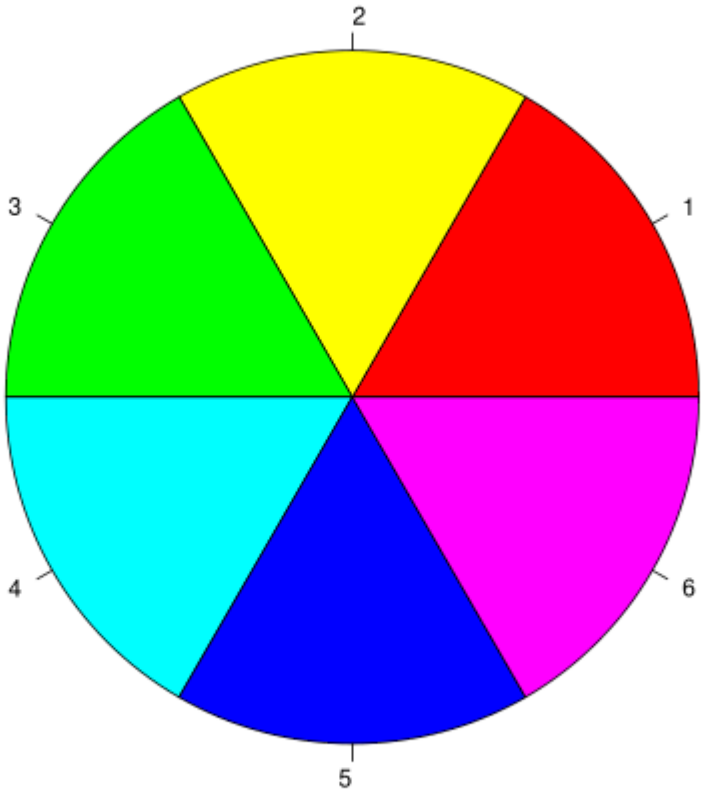
Here an example:

````
```{.pipe pipe=R label=rfig fig=true eval=true fig.width=800 fig.height=600}
pie(rep(1,each=6),col=rainbow(6))
```
````

And here the output.

```
pie(rep(1,each=6),col=rainbow(6))
```

```
> pie(rep(1,each=6),col=rainbow(6))
```



Now the next chunk of code which shows that the variable *x* which was defined a few chunks before still exists in the current session- Using the option `echo=false` we can as well hide the code chunk and only display the output:

```
'```{.pipe pipe=R results=show echo=false}
print(x)
'```
```

Here the output:

```
> print(x)
[1] 3
```

Now again an other chunk of code where we increment x by one and then print it out:

```
> x=x+1
> print(x)
[1] 4
```

Variable values of the R session can be as well displayed inline of the normal text using single back-ticks. Here an example:

```
Now let's show what x is. The variable x has a value of: `r x`.
```

And here the output:

Now let's show what x is. The variable x has a value of: 4.

Let' now display a data frame as a table using the `df2md` function which is provided by tmdoc:

Here the code chunk:

```
'```{.pipe pipe=R results=asis echo=false}
df2md(head(iris))
'```
```

And here the output:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

The advantage of this pipe based approach to the shell based approach, shown before, for executing R code is, that the application status is continued into the next code chunk. See here:

```
'```{.pipe pipe=R results=show echo=false}
ls()
print(x)
'```
```

And here the output:

```
> ls()
[1] "df2md" "iris"  "x"
> print(x)
[1] 4
```

So variable x still exists as well in the new code chunk even it was declared in a chunk earlier. This allows comprehensive analysis in statistical reports. Usually you should use R packages like knitr and rmarkdown for this purpose, however they are much more demanding. If you install these packages using your package manager you might fill up your hard disk with 2GB of files, whereas you can accomplish this with the little **tmdoc** binary by a single around 200KB large file.

To be compatible with R's knitr code as well the following syntax is supported:

```
'```{r results=show echo=false}
y=3
```

````
print(y)
```
````

And here the output:

```
> y=3
> print(y)
[1] 3
```

As Knitr and RMarkdown are using quotes for strings and uppercased TRUE and FALSE with code chunks options separated by commas, as well such code chunks are supported:

````
```{r results="show",echo=FALSE}
y=4
print(y)
```
````

And here the output:

```
> y=4
> print(y)
[1] 4
```

Let's finish the session with a little loop where continuation lines should be rendered with '+' signs:

````
```{r results=show echo=false}
for (i in 1:4) {
    print(i)
}
```
````

```
> for (i in 1:4) {
+     print(i)
+ }
[1] 1
[1] 2
[1] 3
[1] 4
```

Here an example for a code chunk which has an error:

```
x=2
rm(x)
print(x)
```

```
> x=2
> rm(x)
> print(x)
Fehler: Objekt 'x' nicht gefunden
```

nd here we try to load a non-existing library:

```
library(dummy2025)
x=1
print(x)
```
```

```
> library(dummy2025)
Fehler in library(dummy2025) : es gibt kein Paket namens ‘dummy2025’
> x=1
> print(x)
[1] 1
```

Python Code Evaluation

As we can use R we can as well use Python in a similar way using the pipe code chunk. Here an example:

```
def hello():
    print("Hello Python World!\n")

hello()
x=2
```

```
Hello Python World!
```

Again we can use variables declared in the previous code chunks:

```
print(x)
```

```
2
```

We can as well hide the code and only display the session data using the option `echo=false`.

```
3
```

It is as well possible to embed inline python code.

```
So the value of Pythons x is `py print(x)`.
```

Output:

So the value of Pythons x is 3.

To be more compatible with other literature programing tools as well the following syntax where the code chunk starts with py is supported:

````
'```{py results=show echo=false}
y=3
print(y)
'```
````

And here the output:

```
3
```

And now let's check for an error:

```
x1=2
print(x2)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    print(x2)
          ^^
NameError: name 'x2' is not defined. Did you mean: 'x'?
```

Let's check if we can continue to work in the shell:

```
print(x1)
```

```
2
```

In earlier versions there was a problem when computations took longer as the Python pipe then mixed up then the input and the output, we fixed that now by only supporting output but not terminal like input lines. Here an example that a longer lasting computation does not harm the output.

```
import time
x=9
time.sleep(4)
print(x)
```

```
9
```

The value of Pythons x is now 9! Should be 9!

You can as well create plots using the matplotlib library for Python like this:

Here an example:

```
' ```{py results="hide"}
import matplotlib.pyplot as plt
x = [0, 2, 4, 6, 8]
y = [0, 4, 16, 36, 64]

fig, ax = plt.subplots()
ax.plot(x, y, marker='o', label="Data Points")

ax.set_title("Basic Components of Matplotlib Figure")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
plt.savefig('pyexample.svg')
' ```

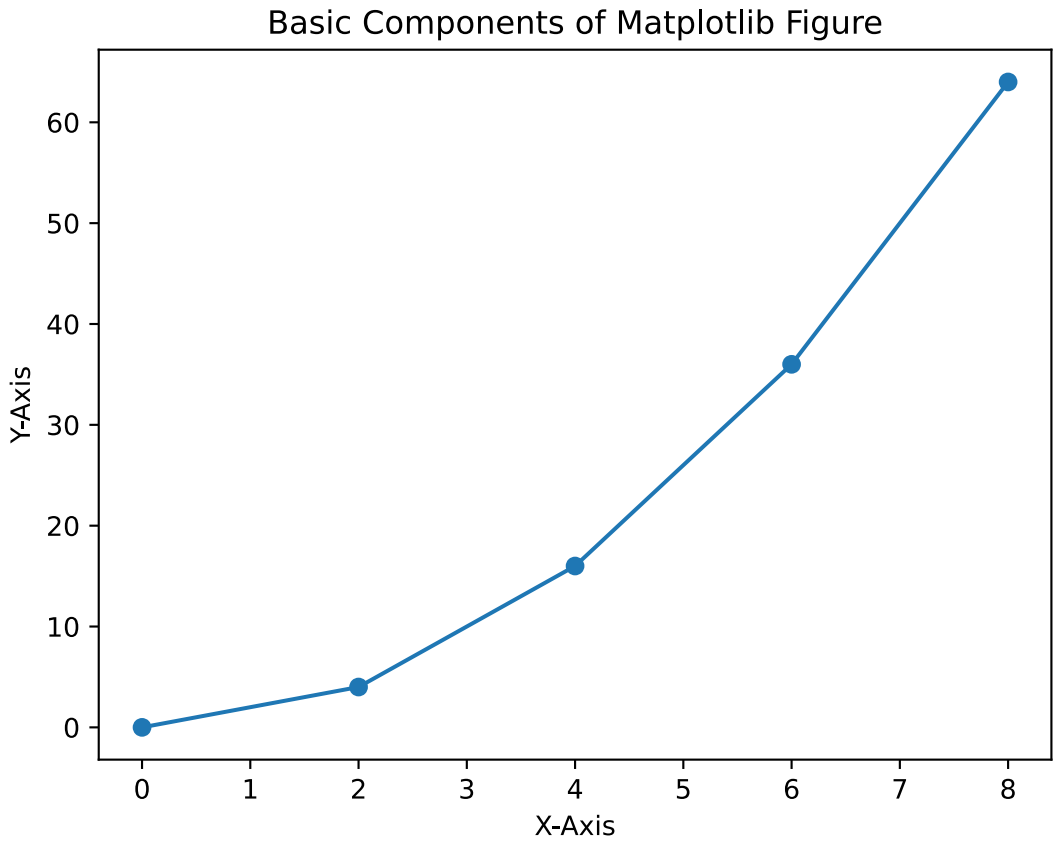
![](pyexample.svg)
```

And here the ouput:

```
import matplotlib.pyplot as plt
x = [0, 2, 4, 6, 8]
y = [0, 4, 16, 36, 64]

fig, ax = plt.subplots()
ax.plot(x, y, marker='o', label="Data Points")

ax.set_title("Basic Components of Matplotlib Figure")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
plt.savefig('pyexample.svg')
```



If you setup the matplotlib library as shown above you can as well get rid of the plot.savefig call by setting the code chunk option `fig=true`. Here an example:

```
import numpy as np

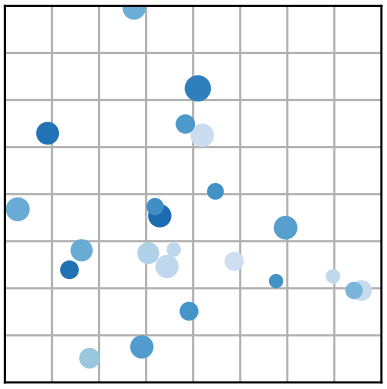
plt.style.use('_mpl-gallery')

# make the data
np.random.seed(3)
x = 4 + np.random.normal(0, 2, 24)
y = 4 + np.random.normal(0, 2, len(x))
# size and color:
sizes = np.random.uniform(15, 80, len(x))
colors = np.random.uniform(15, 80, len(x))

# plot
fig, ax = plt.subplots()

ax.scatter(x, y, s=sizes, c=colors, vmin=0, vmax=100)

ax.set(xlim=(0, 8), xticks=np.arange(1, 8),
       ylim=(0, 8), yticks=np.arange(1, 8))
```



Octave Code Evaluation

As we can use R or Python we can as well use an other statistical programming language, Octave an open source Matlab clone, in a similar way using the pipe code chunk. Here an example:

````
```{.pipe pipe=octave}
str = 'Hello Octave World!';
disp(str);
```
````

And here the output:

```
str = 'Hello Octave World!';
disp(str);
```

```
octave> str = 'Hello Octave World!';
octave> disp(str);
Hello Octave World!
```

Now comes the next code block where we have two *disp* statements to display the value of a variable:

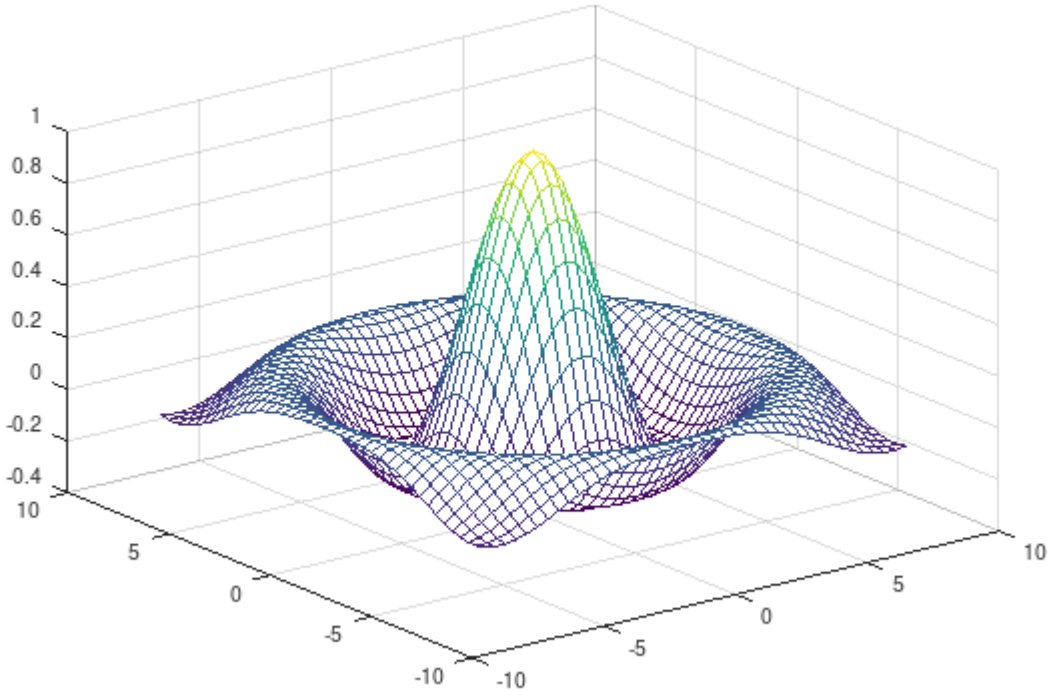
```
x=1;
disp(x);
y=2;
disp(y);
```

```
octave> x=1;
octave> disp(x);
1
octave> y=2;
octave> disp(y);
2
```

Now let's do a plot using the *figure* and the *print* methods:

```
aux=figure('visible','off');
tx = ty = linspace (-8, 8, 41);
[xx, yy] = meshgrid (tx, ty);
r = sqrt (xx .^ 2 + yy .^ 2) + eps;
tz = sin (r) ./ r;
mesh (tx, ty, tz);
print(aux, "mesh-octave.png", "-dpng", "-S600,400");
```

```
octave> aux=figure('visible','off');
octave> tx = ty = linspace (-8, 8, 41);
octave> [xx, yy] = meshgrid (tx, ty);
octave> r = sqrt (xx .^ 2 + yy .^ 2) + eps;
octave> tz = sin (r) ./ r;
octave> mesh (tx, ty, tz);
octave> print(aux, "mesh-octave.png", "-dpng", "-S600,400");
```



Now let's try inline Octave. The value of x is 1!

Using the option `fig=true` we can avoid adding the to code lines which open the figure and the print command to save the image and use the current chunk name.

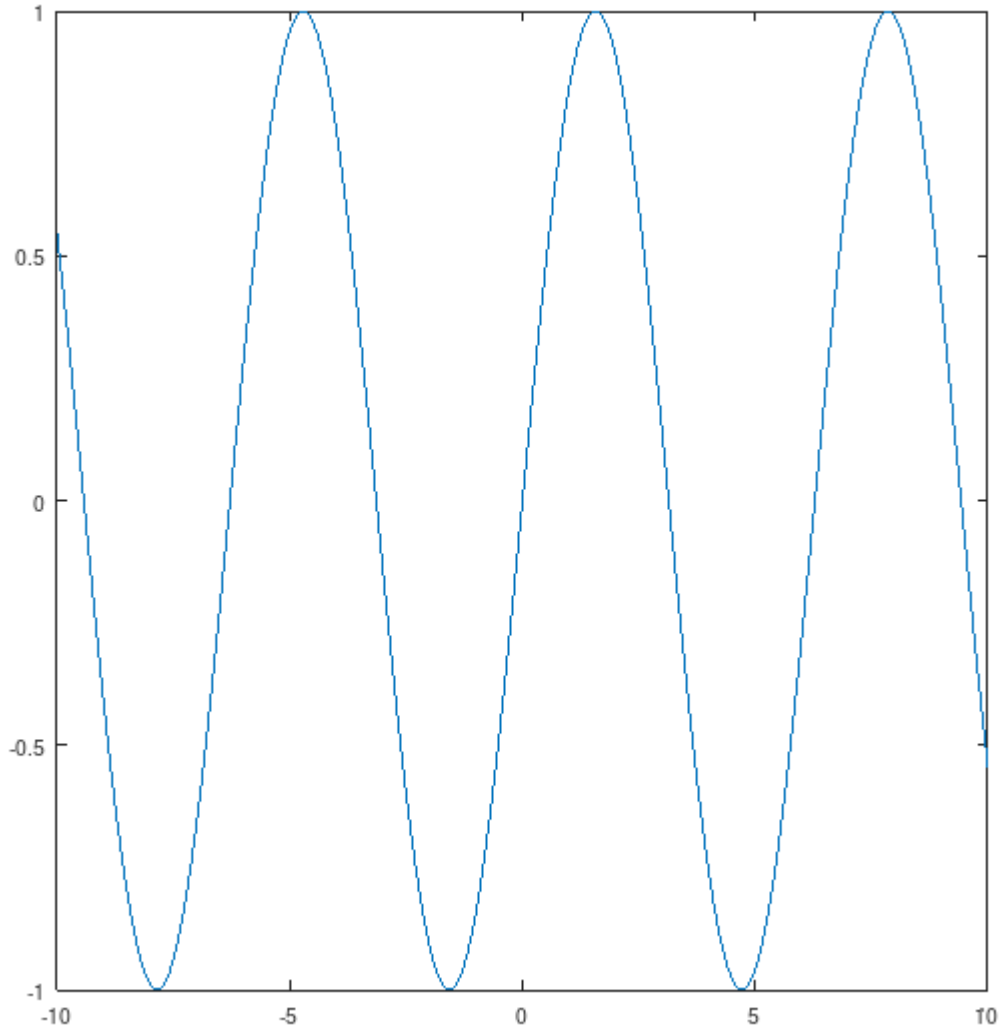
Here an example:

```
'```{.pipe pipe=octave label=oct1 fig=true eval=true}
x = -10:0.1:10;
plot (x, sin (x));
'```
```

And here the output:

```
x = -10:0.1:10;
plot (x, sin (x));
```

```
octave> x = -10:0.1:10;
octave> plot (x, sin (x));
```



Music Chord Sheets

Below we have an example for a tcrd sheet.

```
'``{.tcrd echo=FALSE}
[Dm]Are you going to [C]Scarborough [Dm]Fair?
[F]Parsley, [Dm]sage, rose [F]mary [G]and [Dm]thyme
Remember [F]me to one who [C]lives there
[Dm]He once [C]was a true love of [Dm]mine
'``
```

And here is the output:

```
Dm              C           Dm
Are you going to Scarborough Fair?
F        Dm         F    G   Dm
Parsley, sage, rose mary and thyme
         F             C
Remember me to one who livesxb there
Dm      C                  Dm
He once was a true love of mine
```

Using the chode chunk option argument `transpose=2` you transpose the song by two half tone steps above like so:

````
```{.tcrd echo=FALSE transpose=2}
[Dm]Are you going to [C]Scarborough [Dm]Fair?
[F]Parsley, [Dm]sage, rose [F]mary [G]and [Dm]thyme
Remember [F]me to one who [C]lives there
[Dm]He once [C]was a true love of [Dm]mine
```
````

And here is the output:

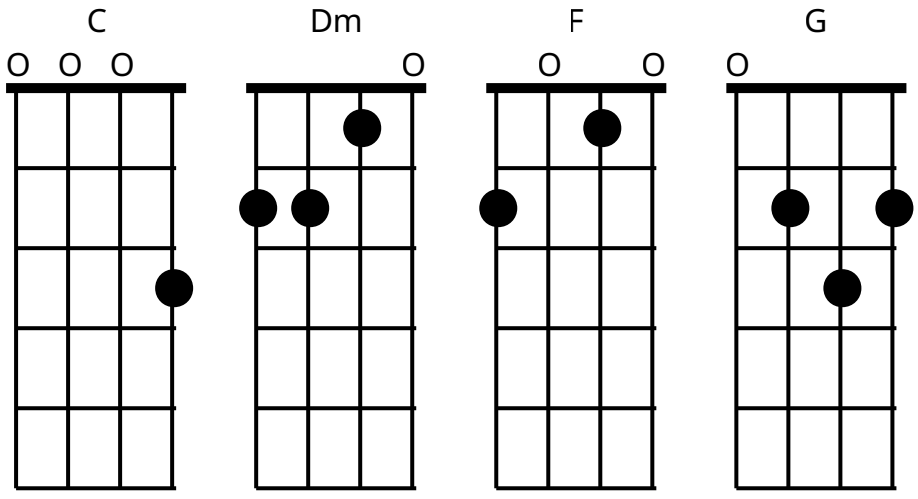
```
Em               D           Em
Are you going to Scarborough Fair?
G        Em         G    A   Em
Parsley, sage, rose mary and thyme
         G             D
Remember me to one who livesxb there
Em      D                  Em
He once was a true love of mine
```

Let's now add a few Ukulele Chord figures:

````
```{.tcrd chord=true,echo=true,fig=true,out.width=300}
C  0003
Dm 2210
F  2010
G  0232
```
````

And here the output:

```
C  0003
Dm 2210
F  2010
G  0232
```



tblocks - diagrams from Markdown code

Sice version 0.20.0 there is as well support for a diagram tool which just used ordinary Markdown as input to create flowcharts or diagrams.

Here some input:

````
'``{.tblocks}
---
mode: "inout"
---
## Input

MD file

Markdown document with
headers and normal text
as input for your diagram.

## Tcl-Script

tblocks.tcl

The standalone Tcl script
which can create several
type of diagrams such
as this.

## Output

SVG file

A SVG graphics file which
can be further converted
to other file types like
 PDF or SVG.
'``
````

And here the output:

And here an other diagram type, the blocks type:

````
'``{.tblocks label=blocks}
---
mode: "blocks"
---
## Call by Value
Concept:
    Clones the data.

`int add(int x, int y);`
`add(x,y);`

Safe but inefficient for large
data structures.

## Pointers
Concept:
    Passes memory address.

`void swap(int *x, int *y);`
`swap(&x,&y);`

Requires eight '*' and two '&' symbols
for resolving the address issues.

## References
Concept:
    Creates an alias.

`void swap(int &x, int &y);`
`swap(x,y);`

Requires only 4 '&' and only in the
argument lists. Clean function call.
````

```
References can't be reassigned.
```

And here the output:

The created Markdown and image files should be in subfolder images in your current directory. They are automatically included.

For more examples consult the Github project page of tblocks.

Code Chunk Options

The following options for Tcl code chunks which are declared with as *{.tcl}* are currently available:

- *echo*: Should the Tcl code be shown, true or false, default: true.
- *eval*: Should the code chunk be evaluated, default: true
- *fig*: Does the code chunk produces a graphic, default: false for Tcl code If this feature is used there must be a `figure` procedure with a filename argument which writes the figure.
- *include*: Should the created figure automatically displayed, default: false. It is as well recommended to give the chunk a label, which will be used to create the image filename.
- *label*: Chunk label, helps in debugging and file naming if graphics files are written. default: chunk-nn where nn is the current chunk.
- *results*: Should the output of the Tcl code evaluation be shown, either "hide", "asis" or "show", default: "show".

The following code chunk options are available for Shell / Cmd code which are declared with *{.cmd}* or *{.shell}* as chunks:

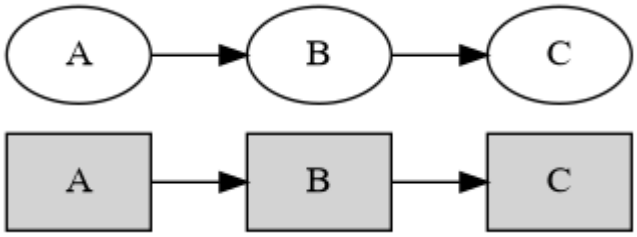
- *cmd*: the command line to process the input in the code chunk, the following place holders are available
  - *%i*, *%f* - the input filename
  - *%o* - the output filename
  - *%b* - the basename of the input filename
- *chunk.ext* - file filename extension for the code chunk, the default is txt, but for some tools like for instance a compiler an extension used usually for that programming language should be used
- *echo*: Should the Input for the command line tool be shown, either true or false, default: true.
- *ext*: file extension for the output file, either a image extension like png or `svg` or a text extension like `txt` for showing text output, default: png
- *fig*: Does the code chunk produces a graphic, default: true
- *label*: Chunk label, helps in debugging and file naming if graphics files are written. default: chunk-nn where nn is the current chunk.
- *results*: Should the output of the Tcl code evaluation be shown, either "hide", "asis" or "show", default: "show".
- *include*: Should the created figure automatically displayed, default: true. If set to false it is as well recommended to give the chunk a label, which will be used to create the image filename.

If the same input should be used with an other command line invocation, as code chunk should be filled with the text string *LASTFILE*.

Here an example:

```
```{.cmd cmd="dot -Tpng -o%o %i" echo=false}
digraph g { rankdir="LR"; A -> B -> C }
```
```{.cmd cmd="dot -Tpng -Nshape=box -Nstyle=filled -o%o %i" echo=false}
LASTFILE
```
```

Here the output:



The following code chunk options are available for Kroki code chunks which are declared with *{.kroki}* :

- *dia*: The diagram type such as ditaa, plantuml, graphviz, pikchr etc, default: ditaa
- *echo*: Should the Input for the command line tool be shown, either true or false, default: true.
- *ext*: The file extension, such as pdf, png or svg, default: png
- *fig*: Does the code chunk produces a graphic, default: true
- *fig.path*: The folder where to store the images, default . (dot, current working directory)
- *label*: Chunk label, helps in debugging and file naming if graphics files are written. default: chunk-nn where nn is the current chunk.
- *results*: Should the output of the Tcl code evaluation be shown, either "hide", "asis" or "show", default: "show".
- *include*: Should the created figure automatically displayed, default: true. If set to false it is as well recommended to give the chunk a label, which will be used to create the image filename.

Summary

The `tmdoc::tmdoc` package provides a literate programming proccessor which extracts and evaluates Tcl, Shell, LaTeX equations or Diagram code within code chunks of Markdown documents. The code chunks are evaluated by the `tmdoc` processor and replaced with their output. The code or input and the output can be shown or hidden due to specific configuration options for the code chunk.There is as well the possibility to display images created by Tcl within the document and inline evaluation of short Tcl commands within backtick where the first backtick is followed by the character sequence like here: `'tcl set x 1'` (replace single quotes with backticks). Further more there exists code to include other files into the current document and to work with references stored in BibTeX files.

Code chunks for Tcl code start on lines with three backticks and the character sequence tcl after a curly brace. They end at the next line which has again three backticks (replace the initial single quote with one backticks).

```
'``{.tcl}
set x 1
'``

or Pandoc style

'`` {.tcl}
set x 2
'``
```

Installation

To install the command line application, just download the latest bin file from the release page of the Github project, rename it as tmdoc, make it executable and move to a directory belonging to your PATH variable.

Here an example:

```
wget https://github.com/mittelmark/tmdoc/releases/download/v0.18.0/tmdoc-0.18.0.bin \
    -O ~/.local/bin/tmdoc
chmod 755 ~/.local/bin/tmdoc
tmdoc --version
### 0.18.0
```

Alternatively you can install the latest version and the mndoc Markdown to HTML converter together using the following install command in your terminal:

```
/bin/bash -c \
 "$(curl -fsSL https://github.com/mittelmark/tmdoc/releases/latest/download/install-tmdoc.sh)"
```

Thereafter you should check your installation using the `--version` flag for bot applications.

```
tmdoc --version
mndoc --version
```

To do then a full document conversion from R-markdown or T-Markdown files you can then do the following:

```
tmdoc infile.Rmd - --toc true | mndoc - outfile.html --css tmdoc.css
```

This should convert the given *infile.Rmd* to the output file *outfile.html* creating as well toc-file for infile.Rmd which can included in that file and the HTML use will use the given stylesheet. Please not that all local image and stylesheet will be embedded per default into the HTML document.

To install the library copy the folder *modules/tmdoc* to a folder belonging to your TCLLIBPATH variable. If this variable does not exists you can set it for instance to `~/.local/lib/Tcl8.6` and place the code therein.

See Also

- filter-abbrev
- filter-julia
- filter-mtex
- filter-python
- filter-r
- filter-tcl
- filter-tcrd
- tmdoc-package
- tmdoc-application
- mndoc package and application

Processing Information

For generating the document the following Tcl version and Tcl packages were used:

- Tcl/Tk 8.6.16
- tmdoc::tmdoc 0.20.1

Document was processed at: 2026-10-29 22:15

Document processing was done within: 21095 milliseconds.

Author and Copyright

Detlef Groth, University of Potsdam, Germany

License

```
BSD 3-Clause License

Copyright (c) 2022-2025, Detlef Groth, University of Potsdam

All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
   list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice,
   this list of conditions and the following disclaimer in the documentation
   and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its
   contributors may be used to endorse or promote products derived from
   this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```